

PRACTICAL – 08

Name: Anthony Nissy

Roll No: 2520030064

Section: 8

EXPERIMENT – Dynamic Memory Allocation and Copy-on-Write

PART A: Dynamic Memory Allocation Using malloc(), calloc(), realloc(), and free()

Aim

To develop a C program using malloc(), calloc(), realloc(), and free(), and identify memory leaks using **Valgrind**.

Objectives

- To understand dynamic memory allocation.
- To use malloc(), calloc(), realloc(), and free().
- To understand heap memory.
- To monitor allocated memory.
- To detect memory leaks using Valgrind.

Theory

Dynamic memory is allocated from the **heap** during program execution. It allows programs to allocate memory according to their requirements.

Function	Purpose
malloc()	Allocates uninitialized memory
calloc()	Allocates memory and initializes it to zero
realloc()	Changes the size of allocated memory
free()	Releases allocated memory

Memory Leak

A memory leak occurs when dynamically allocated memory is not released after use. Valgrind is used to detect memory leaks and invalid memory operations.

Algorithm

1. Allocate memory for 5 integers using malloc().
2. Store and display values.
3. Allocate memory using calloc().
4. Display the zero-initialized values.
5. Increase the malloc() memory using realloc().
6. Store and display 10 values.
7. Release memory using free().
8. Run the program using Valgrind.
9. Check the memory-leak report.

C Program

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
        printf("malloc failed\n");
```

```
    return 1;
}

for (i = 0; i < 5; i++)
    a[i] = i + 1;

printf("malloc values: ");
for (i = 0; i < 5; i++)
    printf("%d ", a[i]);

printf("\n");

int *b = (int *)calloc(5, sizeof(int));

if (b == NULL)
{
    printf("calloc failed\n");
    free(a);
    return 1;
}

printf("calloc values: ");
for (i = 0; i < 5; i++)
    printf("%d ", b[i]);

printf("\n");

a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

for (i = 5; i < 10; i++)
    a[i] = i + 1;
```

```
printf("After realloc: ");  
for (i = 0; i < 10; i++)  
    printf("%d ", a[i]);  
  
printf("\n");  
  
free(a);  
free(b);  
  
printf("Memory released successfully.\n");  
  
return 0;  
}
```

Compilation and Valgrind

```
gcc practical08a.c -o practical08a  
./practical08a
```

For Valgrind:

```
valgrind --leak-check=full ./practical08a
```

Expected Output

```
simra@simran315:~$ nano cow.c
simra@simran315:~$ gcc cow.c -o cow
simra@simran315:~$ gcc cow.c -o cow
simra@simran315:~$ ./cow
Parent PID: 1521
Memory allocated: 50 MB

Child PID: 1522
Child created. Memory is initially shared.
Parent created child PID: 1522
Parent waiting...
Child modifying memory...
Child modified the memory.
```

malloc values: 1 2 3 4 5

calloc values: 0 0 0 0 0

After realloc: 1 2 3 4 5 6 7 8 9 10

Memory released successfully.

Valgrind:

All heap blocks were freed -- no leaks are possible

Observation

- malloc() allocated memory for 5 integers.
- calloc() allocated and initialized 5 integers to zero.
- realloc() increased the memory to hold 10 integers.
- free() released the allocated memory.
- Valgrind showed that there were no memory leaks.

Result

The program successfully demonstrated malloc(), calloc(), realloc(), and free(). Valgrind was used to verify proper memory management.

PART B: Copy-on-Write Using fork()

Aim

To demonstrate **Copy-on-Write (COW)** behavior after fork() and analyze memory usage before and after modifying data in the child process.

Objectives

- To understand Copy-on-Write.
- To understand the behavior of fork().
- To observe memory sharing between parent and child.
- To analyze memory after modification by the child.

Theory

When fork() creates a child process, the parent and child initially have separate virtual address spaces, while Linux can allow them to **share the same physical memory pages**.

When the child modifies a shared page, Linux creates a **private copy** of that page for the child. This mechanism is called **Copy-on-Write**.

Before fork()

Parent

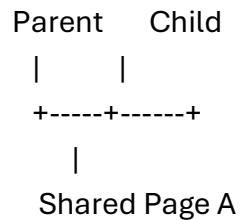
|

Memory

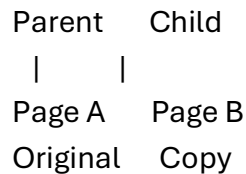
|

Page A

After fork()



After Child Modifies



Algorithm

1. Allocate 50 MB of memory.
2. Fill the memory with data.
3. Display the parent PID.
4. Create a child using fork().
5. The parent and child initially share physical pages.
6. Display the child PID.
7. Modify the memory in the child.
8. The operating system creates private copies of modified pages.
9. Observe the change in memory usage.
10. Parent waits for the child using wait().
11. Release memory using free().

C Program

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (50 * 1024 * 1024)

int main()
{
    char *memory = malloc(SIZE);

    if (memory == NULL)

        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < SIZE; i += 4096)
        memory[i] = 'A';

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated: 50 MB\n");

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        free(memory);
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild PID: %d\n", getpid());
        printf("Child created. Memory is initially shared.\n");

        sleep(10);

        printf("Child modifying memory...\n");
```

```
    for (int i = 0; i < SIZE; i += 4096)
        memory[i] = 'B';

    printf("Child modified the memory.\n");

    sleep(20);

    free(memory);
    exit(0);
}
else
{
    printf("Parent created child PID: %d\n", pid);
    printf("Parent waiting...\n");

    sleep(30);

    wait(NULL);

    free(memory);
}

return 0;
}
```

Compilation and Execution

```
gcc practical08b.c -o practical08b
./practical08b
```

Expected Output

```
Parent PID: 1234
Memory allocated: 50 MB
```

Parent created child PID: 1235

Parent waiting...

Child PID: 1235

Child created. Memory is initially shared.

Child modifying memory...

Child modified the memory.

The PID values will be different for each execution.

Observation

Before Child Modification

- Parent and child can share the same physical pages.
- No complete copy of the memory is immediately required.

After Child Modification

- The child writes to the shared pages.
- Linux creates private copies of the modified pages.
- The parent continues using the original pages.
- The child's private memory usage increases.

Role of wait()

wait(NULL) makes the parent wait until the child process finishes.

Role of 4096

The program uses 4096 bytes as a typical Linux memory-page size. Writing once per page helps demonstrate Copy-on-Write across many pages.

Result

The Copy-on-Write behavior of `fork()` was successfully demonstrated. Initially, the parent and child can share physical memory pages. When the child modifies the memory, Linux creates private copies of the affected pages.

CONCLUSION

The experiment demonstrated two important Linux memory-management concepts:

1. **Dynamic Memory Allocation** using `malloc()`, `calloc()`, `realloc()`, and `free()`, with Valgrind used to check memory leaks.
2. **Copy-on-Write** after `fork()`, where memory pages are initially shared and copied only when one process modifies them.